

Cẩm nang đọc source và lần theo pipeline VideoLevel

Từ điểm vào demo, CLI và API đến các hàm native C++

Tài liệu kỹ thuật tiếng Việt

3 tháng 10 năm 2026

Cập nhật 03/10/2026. Tài liệu được viết lại theo cây mã nguồn hiện tại. Repository chỉ còn **một lỗi media** là thư viện C++ `native/src/cavlc_stream.cpp`; nhánh Python full-video cũ đã bị gỡ. Kênh dùng **giao thức v2** (khóa con HKDF, frame version `0x02`, làm trắng bit bằng keystream XOR) và **một giao thức duy nhất theo đoạn IDR**. Demo chỉ còn `demo/terminal_demo.py`; endpoint `/api/v1/jobs/verify` đã chạy thật (trích mù + Groth16 bắt buộc). Nguồn đối chiếu: `doc/he_thong_hoat_dong.md`, `README.md`, `demo/README.md` và chính mã nguồn.

Mục lục

1	Mục đích và cách dùng tài liệu	3
1.1	Ba lớp kết quả cần phân biệt	3
1.2	Một khác biệt kiến trúc ảnh hưởng đến giao thức	3
2	Sơ đồ hoạt động tổng thể	3
2.1	Ba điểm vào và nơi chúng gặp nhau	3
2.2	Luồng gọi theo thứ tự trong demo	4
3	Cách đọc cấu trúc repository	5
4	Điểm vào demo: <code>demo/terminal_demo.py</code>	6
4.1	Session và các helper dùng chung	6
4.2	Các module demo và hàm chính	6
5	Pha prover: từ message đến payload mang proof	8
5.1	Circuit hiện đang chứng minh điều gì	8
5.2	Ví dụ byte-for-byte với demo	8
5.3	Mã hóa/nén proof	9
5.4	Xác minh Groth16	9
6	Đường native: đọc H.264 và liệt kê ứng viên dấu CAVLC	9
6.1	Tại sao lật đúng một bit dấu giữ parser đồng bộ	9
6.2	Annex-B, EBSP và RBSP	9
6.3	Các kiểu dữ liệu C++ giữ kết quả parser	9
6.4	Call chain C++ từ Annex-B đến ứng viên	10
6.5	Ví dụ một ứng viên	11

7	Kênh xác thực v2: khóa con, lịch, khung và làm trắng	11
7.1	Khóa con HKDF	11
7.2	Lịch có khóa	11
7.3	Khung xác thực và làm trắng	12
7.4	Giao thức theo đoạn IDR	12
7.5	Thông báo lỗi native	13
8	CLI native và công cụ soi	14
8.1	zkstego_blind_bits	14
8.2	zkstego_inspect và lịch tham chiếu Python	15
9	Verifier: từ video stego đến quyết định	15
9.1	Trích mù native	15
9.2	Unpack và verify Groth16	16
9.3	Verifier phía API	16
10	API: hàm nào nhận request, hàm nào chạy job	16
10.1	Lỗi job chung src/api/app.py	16
10.2	Ứng dụng native src/api/native_handlers.py	17
11	Các module hỗ trợ ngoài đường embed/extract	18
11.1	Bộ lập lịch realtime	18
11.2	Chứng chỉ khóa và manifest	18
11.3	Giao diện trust thử nghiệm	18
12	Kiểm thử và tái lập	18
13	Demo đã chạy: artifact và cách suy luận	19
14	Các giới hạn cần nhớ khi đọc code	20
15	Lộ trình đọc source cho người mới tiếp nhận	20
16	Tra cứu nhanh theo câu hỏi debug	20
17	Kết luận vận hành	21

1 Mục đích và cách dùng tài liệu

Tài liệu này là bản đồ đọc code của repository VideoLevel. Mục tiêu là giúp người đọc lần theo một dữ liệu cụ thể qua từng lớp: điểm vào (demo, CLI, API), bộ sinh proof Groth16, CLI native, thư viện C++ phân tích CAVLC, lịch nhúng có khóa, khung xác thực, rồi ngược lại ở phía bên nhận. Mỗi phần nêu file, hàm, đầu vào, đầu ra, tác dụng lên pipeline và bước kế tiếp dùng kết quả đó. Mọi tên file và tên hàm trong tài liệu đã được đối chiếu với mã nguồn ngày 03/10/2026. Đây là hướng dẫn source, không phải đặc tả cho mọi profile H.264.

Điểm quan trọng đầu tiên: chỉ có **một** đường xử lý video. Demo, HTTP job và WebSocket đều gọi cùng tiến trình `zkstego_blind_bits`, và tiến trình này gọi cùng các hàm trong `native/src/cavlc_stream.cpp`. Python chỉ điều phối: sinh proof, đóng gói payload, quản lý job/stream và giải thích trong demo. Python có một bản tham chiếu byte-for-byte của lịch và khung (`src/native_blind_contract.py`) để kiểm thử chéo, nhưng bản này không nhúng hay trích video.

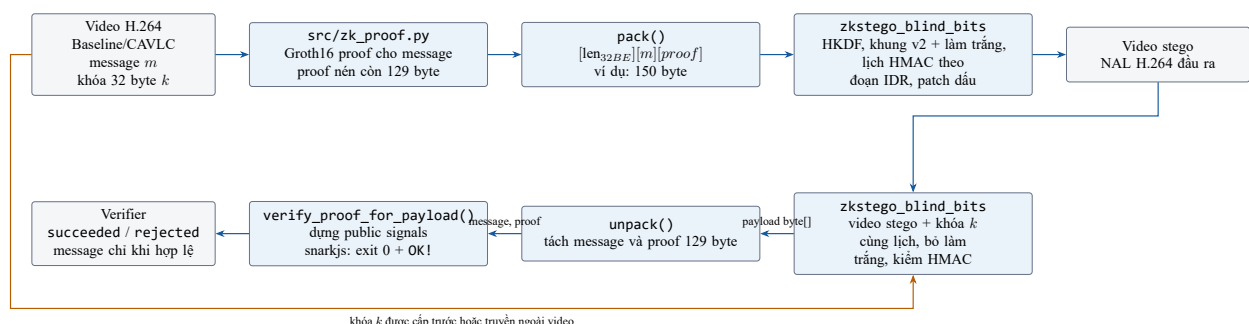
1.1 Ba lớp kết quả cần phân biệt

- **Thành công cú pháp video:** FFmpeg đọc và giải mã stream mà không báo lỗi nghiêm ngặt. Điều này chưa đo PSNR, SSIM hay chất lượng cảm nhận.
- **Thành công khôi phục payload:** extractor native lấy đúng các byte đã đóng gói từ đầu hệ số CAVLC và tag HMAC của khung khớp. Điều này cần đúng khóa 32 byte và đúng giới hạn `max_bits_per_IDR`.
- **Thành công mệnh đề ZK:** `snarkjs groth16 verify` trả mã thoát 0 và in OK! trên đúng public signals. Kết quả này không tự chứng minh nguồn gốc camera, danh tính người ký hay rằng video không bị xử lý ở nơi khác.

1.2 Một khác biệt kiến trúc ảnh hưởng đến giao thức

Video stego mang thông điệp và proof, nhưng verifier vẫn cần khóa 32 byte: khóa sinh lại các khóa con để tái tạo lịch, bỏ làm trắng, kiểm HMAC, và để `ZKSnarkBridge._build_public_signals()` tính lại commitment. Nếu verifier đã được cấp khóa an toàn từ trước thì kênh media chỉ cần truyền video; nếu chưa, cần một kênh cấp khóa riêng. Endpoint `/api/v1/jobs/verify` hiện thực hiện đầy đủ trích mù → unpack → Groth16 verify ở phía server và chỉ trả message khi proof hợp lệ.

2 Sơ đồ hoạt động tổng thể



Hình 1: Luồng proof-bearing. Khóa là input ngoài video ở cả pha nhúng lẫn trích mù.

2.1 Ba điểm vào và nơi chúng gặp nhau

Điểm vào	Hàm Python/C++ đầu tiên	Gặp lỗi native ở đâu
Demo	demo/terminal_demo.py:: main() → walkthrough()	demo/native_io.py:: native_embed() / native_extract() chạy zkstego_blind_bits; trace_video() / inspect_segments() chạy zkstego_inspect.
CLI	native/tools/blind_bits. cpp::wmain()/main() → run()	Gọi trực tiếp embed_authenticated_cavlc_ stream_file(), extract_authenticated_ cavlc_stream_file() hoặc các hàm live trong cùng file.
API HTTP	src/api/app.py:: start_embed()/start_ extract()/start_verify()	Handler do src/api/native_handlers. py::create_native_app() inject; invoke() chạy CLI với khóa qua stdin.
API WebSocket	native_handlers.py:: native_stream()	Mở tiến trình con embed-live-auth-stdin hoặc extract-live-auth-stdin.
Soi/kiểm toán	native/tools/inspect.cpp:: run()	Chỉ đọc: decode_baseline_i_ idr_slices(), analyze_cavlc_ stream_file().

2.2 Luồng gọi theo thứ tự trong demo

walkthrough() là kịch bản chính; mỗi session.stage() in một bước trên terminal. Bảng dưới liệt kê đúng thứ tự gọi trong mã.

#	Hàm được gọi	Input chính	Output chuyển tiếp
1	setup() (chỉ khi --setup)	Toolchain CMake, npm, Circom	Binary native, artifact circuit, khóa Groth16 <i>demo</i>
2	preflight()	PATH, circuits/build	Tools, MeasuredBridge; dừng nếu thiếu artifact
3	video_prepare() (choose_input())	File .y4m/video, --frames, --gop, --qp	source.h264 Baseline/CAVLC, encode.stderr
4	make_proof()	Message UTF-8, khóa ngẫu nhiên 32 byte	message bytes, khóa, proof 129 byte, public signals
5	trace_video() + show_video_trace()	source.h264	trace_before.json (NAL, slice, ứng viên)
6	deep_trace.nal_anatomy(), library_header_trace()	Trace + FFmpeg trace_headers	Đối chiếu header (bỏ khi --brief)
7	pack_and_select()	message, proof, khóa, --segments	payload, EmbedPlan, schedule.json

#	Hàm được gọi	Input chính	Output chuyển tiếp
8	<code>native_embed()</code>	cover, payload, khóa, <code>max_bits</code>	<code>stego.h264</code>
9	<code>trace_video() + audit_embedding()</code>	<code>stego.h264</code>	<code>trace_after.json</code> , ứng viên mẫu
10	<code>visual_quality()</code>	source, stego	PSNR Y/U/V, bản đồ ΔY
11	<code>deep_trace.block_anatomy()</code> , <code>embedded_bit_location()</code>	Ứng viên mẫu, ảnh chưa deblock	Giải phẫu một khối, vị trí byte trong file
12	<code>verify_message()</code>	<code>stego.h264</code> , khóa	message khôi phục + proof đã verify
13	<code>negative_cases.run_all()</code>	stego, khóa sai, proof sửa	Các ca từ chối
14	<code>service_steps.run_all()</code>	cùng input	Job HTTP và WebSocket (bỏ khi <code>--no-service</code>)
15	<code>Session.finish()</code>	Trạng thái	<code>transcript.txt</code> , <code>report.json</code>

3 Cách đọc cấu trúc repository

Đường dẫn	Vai trò và ảnh hưởng đến pipeline
<code>native/include/zkstego/cavlc_stream.hpp</code>	Kiểu dữ liệu và interface công khai: SPS/PPS/slice, macroblock, ứng viên, HKDF, lịch, khung, stream encoder/decoder, reader Annex-B tăng dần.
<code>native/src/cavlc_stream.cpp</code>	Lỗi media duy nhất: parser Annex-B/SPS/PPS/IDR/CAVLC, ứng viên đầu, khóa con HKDF, lịch HMAC, khung v2, làm trắng, patch RBSP, chế độ file và chế độ stream.
<code>native/tools/blind_bits.cpp</code>	CLI <code>zkstego_blind_bits</code> : đọc khóa/payload qua stdin, dispatch 5 lệnh, ghi H.264 hoặc payload hex.
<code>native/tools/inspect.cpp</code>	Công cụ soi duy nhất <code>zkstego_inspect</code> : JSON (mặc định), <code>--summary</code> , <code>--macroblock</code> , <code>--segments</code> .
<code>native/tools/cli_io.hpp</code>	I/O dùng chung cho hai tool: đổi số UTF-8 trên Windows, đọc file có giới hạn, ghi file mới không ghi đè.
<code>native/tests/cavlc_stream_tests.cpp</code>	CTest native (<code>zkstego_cavlc_stream_tests</code>).
<code>src/zk_proof.py</code>	Định dạng payload, nén proof 129 byte, ZKSnarkBridge gọi Node/snarkjs để prove/verify.
<code>circuits/payload_verify.circom</code>	Mạch <code>PayloadVerify</code> : commitment SHA-256, ràng buộc bit, giới hạn độ dài (63 321 constraint).
<code>src/native_blind_contract.py</code>	Bản tham chiếu Python của HKDF, lịch, khung, làm trắng và <code>segment_schedule()</code> ; dùng cho test chéo và phân giải thích của demo.
<code>src/api/app.py</code>	Lỗi job chung: settings, xác thực trước body, giới hạn thử sai, hàng đợi, lưu trữ job, endpoint.
<code>src/api/native_handlers.py</code>	Ứng dụng native: handler embed/extract/verify gọi CLI, verify Groth16 bắt buộc, WebSocket <code>/api/v1/stream</code> .

Đường dẫn	Vai trò và ảnh hưởng đến pipeline
src/realtime_cavlc.py	Bộ lập lịch realtime mô phỏng theo đoạn IDR (hàng đợi có giới hạn, epoch, p95).
src/h264_tables.py	Chỉ chứa bảng VLC CAVLC chuẩn cho phần giải thích của demo; không có parser.
src/key_policy.py, src/manifest.py src/trust/	Chúng chỉ khóa Ed25519 có hạn và manifest có chữ ký. Không nằm trên đường embed/extract native. Giao diện trust thử nghiệm: provenance, fingerprint, attestation, watermark receipt, c2pa_bridge.py.
demo/	Demo duy nhất terminal_demo.py và các module deep_trace.py, h264_explain.py, segment_plan.py, negative_cases.py, service_steps.py, native_io.py.
src/runtest/	run_all.py (12 phase), prepare_fixtures.py, các file test_*.py.
benchmark/	Benchmark media/crypto/ZKP, bộ ghi benchmark camera và báo cáo.

4 ĐIỂM VÀO demo: demo/terminal_demo.py

4.1 Session và các helper dùng chung

Mọi bước đều đi qua đối tượng Session trong terminal_demo.py. Thư mục chạy là demo/runs/terminal_<YYYYmmdd_HHMMSS>_<6 hex>/.

Phương thức	Input → output	Vai trò
Session.say()	text, private → terminal + transcript	Giá trị private=True (secret, khóa con) chỉ in ra terminal; transcript ghi dòng che.
Session.stage()	tiêu đề → bước mới	Dừng chờ Enter nếu không có --auto.
Session.run()	label, argv, stdin → CompletedProcess	Chạy tiến trình con, lưu <label>.stdout/.stderr và thời gian.
Session.check()	tên, bool → [PASS]/[FAIL]	Một [FAIL] dừng demo với mã thoát 1.
Session.save()	tên, giá trị → file JSON	Lưu trace, lịch, proof vào thư mục chạy.
Session.finish()	—	Ghi report.json, đóng transcript.

4.2 Các module demo và hàm chính

Hàm	Tác dụng
demo/native_io.py — mọi lời gọi tool native	
native_tool(), find_tools()	Tìm zkstego_blind_bits và zkstego_inspect trong native/build/Release, Debug hoặc native/build.

Hàm	Tác dụng
<code>trace_video()</code>	<code>zkstego_inspect <video> → JSON schema 1 (NAL, slice IDR, ứng viên toàn file).</code>
<code>inspect_segments()</code>	<code>zkstego_inspect <video> --segments MAX → JSON segments-1.</code>
<code>native_embed()</code>	<code>embed-stream-auth stdin; stdin = khóa hex + payload hex.</code>
<code>native_extract()</code>	<code>extract-stream-auth <video> - 4096 MAX; stdin = khóa hex.</code>
<code>verify_groth16()</code>	Gọi <code>snarkjs groth16 verify</code> ; chỉ True khi exit 0 và có OK!; lỗi hạ tầng ném <code>RuntimeError</code> , không bị tính là “từ chối hợp lệ”.
<i>demo/segment_plan.py — dung lượng và lịch theo đoạn</i>	
<code>resolve_max_bits()</code>	Mặc định 64 bit/IDR; nếu không truyền cờ và clip quá ngắn thì nâng lên giá trị nhỏ nhất đủ dung lượng.
<code>segment_capacity(), show_allocation()</code>	Tính $\sum \min(\text{max_bits}, \text{ứng viên})$; in đoạn nào mang khoảng bit nào của khung.
<code>file_candidate()</code>	Đổi định danh ứng viên trong đoạn sang chỉ số NAL trong file.
<i>demo/deep_trace.py và demo/h264_explain.py — giải thích sâu</i>	
<code>ffmpeg_pipeline(), raw_input_view(), encode_loss()</code>	Đọc log verbose của FFmpeg/libx264, hiển thị frame thô, PSNR do nén.
<code>nal_anatomy(), library_header_trace()</code>	Bảng NAL/EPB, đối chiếu header do <code>trace_headers</code> của FFmpeg giải với parser native.
<code>block_anatomy()</code>	Gọi <code>zkstego_inspect --macroblock, decoder_mb_maps()</code> , rồi các hàm <code>h264_explain</code> (<code>parse_i_macroblock_header()</code> , <code>segment_cavlc_block()</code> , <code>dequantize()</code> , <code>inverse_transform()</code> , <code>predict_intra4x4()</code> ...) để tái tạo pixel từng mẫu.
<code>embedded_bit_location(), file_offset_of_rbsp_bit()</code>	Từ thứ hạng HMAC → chỉ số bit khung → trường payload → byte RBSP/EBSP → offset trong file.
<i>demo/negative_cases.py và demo/service_steps.py</i>	
<code>negative_cases.run_all()</code>	Đổi message, proof sai, thiếu proof, sai khóa, lật một bit mang dữ liệu; kiểm tra thông báo lỗi bằng <code>is_authentication_failure()</code> .
<code>service_steps.run_all()</code>	Dùng <code>create_native_app()</code> trong <code>TestClient</code> ; <code>_http_jobs()</code> (embed/extract/verify) và <code>_websocket_stream()</code> .

5 Pha prover: từ message đến payload mang proof

5.1 Circuit hiện đang chứng minh điều gì

Trong `src/zk_proof.py`, message là bytes m và khóa 32 byte là k . `ZKSnarkBridge._build_circuit_input()` tính

$$h = \text{SHA256}(m), \quad c = \text{SHA256}(h||k).$$

Template `PayloadVerify` trong `circuits/payload_verify.circom` nhận $h[256]$, $c[256]$ và `payload_length` làm public signals (tổng 513), còn $k[256]$ là private witness. Ràng buộc gồm: $c = \text{SHA256}(h||k)$; $0 < \text{payload_length} < 10^6$; và từ 03/10/2026 mọi tín hiệu của h , c , k phải là bit, $x(x-1) = 0$, vì thành phần Sha256 của circomlib không tự ép đầu vào về 0/1. Mạch có 63 321 constraint (`ZKSnarkBridge.CONSTRAINT_COUNT`). Proof tạo bằng khóa của mạch cũ không verify được với khóa hiện tại. Circuit không nhận pixel/frame H.264, không chứa hash video hay danh tính camera.

Hàm	Input → output	Tác dụng kế tiếp
<code>_build_circuit_input(m, k)</code>	$m, k \rightarrow$ dict bit của h, c, k và <code>length</code>	Witness generator của Node đọc JSON này.
<code>_compute_witness(input)</code>	Circuit input → file witness tạm	Chạy WASM của mạch; file input chứa khóa bị xóa sau khi có witness.
<code>_snarkjs_prove(witness)</code>	Witness + <code>proving_key.zkey</code> → (proof, public)	Hai dict snarkjs cho bước nén và log.
<code>generate_proof_for_payload(m, k)</code>	$m, k \rightarrow$ (proof, public)	Wrapper đầy đủ; demo gọi qua <code>MeasuredBridge</code> để đo thời gian.
<code>proof_to_bytes(pack(m, proofBytes))</code>	A, B, C → 129 byte $m, \text{proof} \rightarrow [\text{len}_{32BE}][m][\text{proof}]$	Nén proof để nhúng. Blob opaque đưa cho CLI native.
<code>_build_public_signals(m, k)</code>	$m, k \rightarrow$ 513 chuỗi	Verifier tự dựng lại public input.
<code>verify_proof_for_payload()</code>	proof, $m, k \rightarrow$ bool	Dựng public signals rồi <code>verify()</code> → <code>_snarkjs_verify()</code> .

5.2 Ví dụ byte-for-byte với demo

Với `--auto` và không truyền `--message`, `make_proof()` dùng message `Xin chao ZK video`, UTF-8 dài 17 byte. `pack()` ghép:

```
00 00 00 11 | 58 69 6e 20 63 68 61 6f 20 5a 4b 20 76 69 64 65 6f | <129 proof bytes>
length=17                                     message UTF-8
```

Tổng payload là $4 + 17 + 129 = 150$ byte. Khung native v2 thêm $1 + 2 + 16 = 19$ byte, nên khung dài 169 byte, tức 1 352 bit phải nhúng. Message tối đa là $4096 - 4 - 129 = 3963$ byte vì lệnh `stdin` nhận payload tối đa 4096 byte.

5.3 Mã hóa/nén proof

`proof_to_bytes()` ghi tọa độ x của A, hai phần tử $\mathbb{F}_{p^2}$ của B.x và C.x (mỗi phần 32 byte big-endian), cộng một byte cờ chứa 3 bit chẵn/lẻ của y : $4 \times 32 + 1 = 129$ byte. `bytes_to_proof()` làm chiều ngược: kiểm tọa độ $< p$, giải phương trình đường cong BN254 để dựng lại y , kiểm điểm nằm trên đường cong; dữ liệu hỏng ném `ValueError`. Không được cắt/gắn proof tùy ý: mọi byte đều đi vào phép giải nén.

5.4 Xác minh Groth16

`_snarkjs_verify()` ghi proof/public tạm rồi chạy `snarkjs groth16 verify`. Kết quả chỉ là `True` khi mã thoát 0 và stdout có dòng kết thúc bằng **OK!**; exit 0 mà không có OK! bị coi là không hợp lệ. Khóa proving/verification là thiết lập giáo dục cục bộ (`circuits/build/DEMO_SETUP_ONLY.txt`), không phải trusted ceremony.

6 Đường native: đọc H.264 và liệt kê ứng viên dấu CAVLC

6.1 Tại sao lật đúng một bit dấu giữ parser đồng bộ

CAVLC residual block mở đầu bằng `coeff_token` mã hóa `TotalCoeff` và `TrailingOnes`. Với mỗi trailing one, stream ghi một `trailing_ones_sign_flag`: bit 0 là +1, bit 1 là -1. Lật bit này chỉ đổi dấu một hệ số; không đổi số hệ số, n_c của các khối sau, bảng VLC được chọn, độ dài mã hay vị trí khối kế tiếp. Vì vậy native patch đúng bit RBSP đó thay vì giải mã frame thành pixel rồi encode lại. Không thể lật tùy tiện một bit trong NAL: nó có thể sửa `coeff_token`, level hoặc header macroblock và làm lệch toàn bộ phần sau.

6.2 Annex-B, Ebsp và Rbsp

`split_annex_b()` quét start code 3/4 byte, cắt elementary stream thành `AnnexBNalUnit` (offset, độ dài start code, header, payload Ebsp). `AnnexBNalUnit::rbsp()` gọi `ebsp_to_rbsp()` để bỏ byte `0x03` đặt sau `00 00`. Parser đọc cú pháp trên RBSP bằng `RbspBitReader` (`read_bits()`, `read_ue()`, `read_se()`). Khi patch, `patch_annex_b_rbsp_plan()` áp các `FixedLengthBitPatch` theo từng NAL, `rbsp_to_ebsp()` chèn lại emulation-prevention byte và `assemble_annex_b()` ghép start code + header + payload. Độ dài bit RBSP không đổi, nhưng độ dài byte Ebsp có thể lệch vài byte.

6.3 Các kiểu dữ liệu C++ giữ kết quả parser

Kiểu trong header	Dữ liệu và consumer tiếp theo
<code>AnnexBNalUnit</code>	Offset, độ dài start code, <code>nal_ref_idc</code> , <code>nal_unit_type</code> , payload Ebsp; <code>is_idr()</code> , <code>rbsp()</code> .
<code>H264BaselineSps</code>	Profile/level, id, kích thước theo MB, trường frame/POC; <code>validated_picture_macroblock_count()</code> giới hạn kích thước ảnh.
<code>H264BaselinePps</code>	Entropy mode (phải là CAVLC), slice group, QP khởi đầu, cờ deblocking.
<code>H264BaselineIdr</code> <code>SliceHeader</code>	first MB, slice type, PPS id, frame/IDR id, QP delta và <code>data_bit_offset</code> đầu slice data.
<code>CavlcCoeffToken</code>	<code>TotalCoeff</code> , <code>TrailingOnes</code> , <code>sign_bit_offsets</code> , offset level; <code>start_bit_offset</code> và n_c chỉ để chặn đoán.

Kiểu trong header	Dữ liệu và consumer tiếp theo
CavlcDecodedLumaBlock	Token, levels, tail (total zeros/runs), hệ số tái dựng. Dùng chung cho luma, LumaDC, chroma DC/AC.
H264BaselineIMacroblockHeader	mb_type, CBP, QP delta, 16 mode Intra4x4, offset residual.
CavlcDecodedIMacroblock	Địa chỉ MB, header, luma_dc, 16 khối luma, 2 chroma DC, 8 chroma AC.
CavlcDecodedIdrSlice	NAL index, SPS/PPS/slice header, danh sách MB, offset trailing bits; đầu vào của bộ thu ứng viên.
CavlcSignCandidate	(NAL index, địa chỉ MB, CavlcResidualCategory, chỉ số khối, offset bit RBSP).
AuthenticatedCavlcStreamSegment	Byte output, candidate_capacity, bits_embedded, session_complete; trả về sau mỗi đoạn.
CavlcStreamSegmentCandidates	Với mỗi đoạn IDR: chỉ số NAL trong file, chỉ số NAL trong input phân tích, danh sách ứng viên.

6.4 Call chain C++ từ Annex-B đến ứng viên

Các hàm dưới đây nằm trong native/src/cavlc_stream.cpp, interface ở native/include/zkstego/cavlc_stream.hpp:

Hàm	Input → output	Ảnh hưởng/consumer
ebsp_to_rbsp() / rbsp_to_ebsp()	Byte vector ↔ byte vector	Chuyển giữa byte NAL và bitstream parser/patcher thao tác.
parse_baseline_sps()	SPS RBSP → SPS	Kích thước, cờ cho slice và MB.
parse_baseline_pps()	PPS RBSP → PPS	Kiểm entropy mode và tham số.
parse_baseline_idr_slice_header()	Slice RBSP + SPS/PPS → header	Chỉ ra data_bit_offset.
parse_baseline_i_macroblock_header()	RBSP + offset → header MB	Quyết định cú pháp residual tiếp theo.
parse_cavlc_coeff_token()	RBSP, bit, n_C → token	Chọn bảng VLC theo n_C ; trả các offset bit đầu.
decode_cavlc_non_trailing_levels()	Token + RBSP → levels	Nhảy qua phần level sau bit đầu.
decode_cavlc_residual_tail() (và biến thể luma/chroma DC)	TotalCoeff, offset → total zeros, runs	Đồng bộ vị trí khối kế tiếp.
reconstruct_cavlc_block()	Levels + runs → hệ số	Điền hệ số theo thứ tự quét.
decode_cavlc_luma_block(), decode_cavlc_luma_ac_block(), decode_cavlc_chroma_dc_block(), decode_cavlc_chroma_ac_block()	RBSP + offset + ngữ cảnh → khối	Chuyên biệt theo loại khối.

Hàm	Input → output	Ảnh hưởng/consumer
<code>decode_cavlc_luma_macroblock()</code> / <code>decode_cavlc_luma_ac_macroblock()</code>	CBP, offset, TotalCoeff hàng xóm → 16 khối	Tính n_C theo hàng xóm đã giải mã.
<code>decode_baseline_i_idr_slice()</code>	RBSP + SPS/PPS → một slice	Lập các MB của I-slice.
<code>decode_baseline_i_idr_slices()</code>	Annex-B → các slice IDR	Theo dõi SPS/PPS theo id; dùng cho inspect, đo dung lượng và chuẩn bị đoạn.
<code>collect_cavlc_trailing_one_sign_candidates()</code>	Các slice → ứng viên	Một ứng viên mỗi khối: dấu trailing-one đầu tiên, theo thứ tự LumaDC, Luma4x4, ChromaDC, ChromaAC.
<code>serialize_cavlc_sign_candidate()</code>	Ứng viên → chuỗi ASCII	Định danh để chấm điểm HMAC.
<code>inspect_baseline_idr_headers()</code>	Annex-B → header/khối đầu IDR	Hàm chẩn đoán, không quyết định lịch.

6.5 Ví dụ một ứng viên

Định danh có dạng `nal_index:macroblock_address:category:block_index:rbp_bit_offset`, ví dụ `24:326:1:6:67801` (category 1 = Luma4x4). Đây là địa chỉ cú pháp, không phải tọa độ pixel. Trong giao thức theo đoạn, `nal_index` tính trong *input phân tích của đoạn* (SPS+PPS+IDR), không phải chỉ số NAL trong file; demo dùng `segment_plan.file_candidate()` để đổi sang chỉ số trong file.

7 Kênh xác thực v2: khóa con, lịch, khung và làm trắng

7.1 Khóa con HKDF

Secret 32 byte không bao giờ được dùng trực tiếp làm khóa HMAC. Hàm nội bộ `derive_cavlc_subkey()` gọi `hkdf_sha256_extract()` và `hkdf_sha256_expand()` (RFC 5869); hàm công khai `hkdf_sha256()` dùng cho test vector:

```
PRK          = HMAC-SHA256(key = "zkstego-cavlc-v2-salt", msg = secret)
schedule_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/schedule", 32)
frame_key    = HKDF-Expand(PRK, "zkstego/cavlc/v2/frame-tag", 32)
whitening_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/whitening", 32)
```

Lộ một khóa con không làm lộ secret hay các khóa con khác. Encoder/decoder xóa secret của caller ngay sau khi dẫn xuất và chỉ giữ khóa con.

7.2 Lịch có khóa

`select_with_schedule_key()` (nội bộ) kiểm từng ứng viên bằng `validate_blind_schedule_candidate()`, từ chối định danh hoặc đích patch (`nal:bit`) trùng lặp, tính

$$\text{score} = \text{HMAC}(\text{schedule_key}, \text{định danh}),$$

rồi `std::partial_sort` theo (score, định danh) để lấy N ứng viên đầu. Nếu N lớn hơn số ứng viên, hàm ném `blind schedule capacity is insufficient`. Các hàm công khai `score_keyed_cavlc_`

`sign_candidate()` và `select_keyed_cavlc_sign_candidates()` nhận secret gốc, tự dẫn xuất `schedule_key` rồi gọi cùng logic.

7.3 Khung xác thực và làm trắng

```
frame      = [0x02][payload_len BE16][payload][tag 16 bytes]
tag        = HMAC-SHA256(frame_key, version || len || payload)[0:16]
keystream  = HMAC(whitening_key, uint64_be(0)) || HMAC(whitening_key, uint64_be(1))
            || ...
embedded[i] = frame_bit[i] XOR keystream_bit[i]      (i global over all segments,
            MSB-first)
```

Hàm (cavlc_stream.cpp)	Vai trò
<code>pack_frame_with_key()</code>	Ghép version 0x02, độ dài BE16, payload, 16 byte đầu của <code>authenticated_frame_tag()</code> . Payload tối đa 65 535 byte theo định dạng.
<code>unpack_frame_with_key()</code>	Kiểm version, độ dài $\leq$ giới hạn và đúng kích thước, so tag theo thời gian hằng; ném lỗi frame <code>version/length/tag</code> .
<code>bytes_to_msb_bits()</code> , <code>msb_bits_to_bytes()</code>	Quy ước bit MSB trước cho cả hai phía.
<code>whitening_block()</code>	Khối keystream thứ $j = \text{HMAC}(\text{whitening_key}, \text{uint64_be}(j))$.
<code>whiten_cavlc_frame_bits()</code>	XOR bit với keystream bắt đầu từ chỉ số bit toàn cục; gọi lại lần nữa để bỏ làm trắng.
<code>pack_authenticated_cavlc_frame()</code> , <code>unpack_authenticated_cavlc_frame()</code> , <code>cavlc_whitening_keystream()</code>	Bản công khai nhận secret gốc; dùng trong CTest và để so với bản Python.

Làm trắng cần thiết vì byte version cố định, trường độ dài nhỏ và cấu trúc proof làm thống kê dấu tại các vị trí được chọn bị lệch. Giao thức v2 không đọc được file stego do phiên bản trước tạo ra.

7.4 Giao thức theo đoạn IDR

Chỉ có một giao thức, dùng chung cho file, job HTTP và luồng live: mỗi đoạn là ngữ cảnh SPS/PPS đã thấy + đúng một IDR; mỗi đoạn mang tối đa `max_bits_per_IDR` bit của khung; chỉ số bit khung chạy liên tục qua các đoạn. Bên nhận phải dùng cùng `max_bits_per_IDR`.

Hàm	Vai trò
<code>update_parameter_set_context()</code>	Giữ đúng một SPS và một PPS; cùng id thì thay, khác id cùng loại thì từ chối.
<code>assemble_cavlc_stream_segment()</code>	Ghép ngữ cảnh SPS/PPS + một IDR thành một đoạn Annex-B.
<code>prepare_cavlc_stream_segment()</code> (nội bộ)	Kiểm đoạn có đúng một IDR và không có P-slice; dựng input phân tích; giải mã slice; thu ứng viên.

Hàm	Vai trò
AuthenticatedCavlc StreamEncoder (constructor)	Dẫn xuất ba khóa con, đóng khung, đổi sang bit, làm trắng toàn bộ một lần; chỉ giữ <code>schedule_key</code> và bit khung đã làm trắng.
AuthenticatedCavlc StreamEncoder:: process_segment()	Chọn min(max_bits, ứng viên) vị trí theo lịch; chỉ patch phần bit khung còn lại; vị trí thừa ở đoạn cuối giữ dấu gốc; gọi <code>patch_annex_b_rbsp_plan()</code> rồi <code>remove_stream_context_prefix()</code> .
complete(), remaining_bits()	Caller biết khi nào khung đã nhúng hết.
AuthenticatedCavlc StreamDecoder:: consume_segment()	<code>extract_keyed_bits_from_candidates()</code> đọc bit dấu theo lịch; bỏ làm trắng với chỉ số toàn cục; sau 24 bit kiểm version và độ dài; đủ bit thì <code>unpack_frame_with_key()</code> . Mọi lỗi chuyển decoder sang <code>failed()</code> (fail-closed).
authenticated_payload()	Trả payload chỉ khi tag đã khớp; nếu chưa: <code>authenticated CAVLC stream is incomplete.</code>
embed_authenticated_cavlc_ stream_file()	Chế độ file: <code>visit_cavlc_stream_file()</code> duyệt NAL; IDR còn cần bit thì qua encoder và <code>single_idr_payload()</code> ; NAL khác và IDR sau khi xong được sao chép nguyên.
extract_authenticated_cavlc_ stream_file()	Chế độ file phía nhận: đưa từng đoạn vào decoder, dừng khi xong.
analyze_cavlc_stream_file()	Góc nhìn kiểm toán: ứng viên của từng đoạn đúng như codec thu được; nguồn của <code>zkstego_inspect --segments</code> .

7.5 Thông báo lỗi native

Mọi lỗi của CLI in error: ... ra stderr và thoát mã 2.

Thông báo	Khi nào / hàm phát sinh
authenticated CAVLC stream version is invalid	Decoder: byte đầu sau khi bỏ làm trắng khác <code>0x02</code> , thường do sai khóa.
authenticated CAVLC stream length exceeds configured maximum	Decoder: trường độ dài vượt giới hạn payload.
authenticated CAVLC frame tag is invalid	<code>unpack_frame_with_key()</code> : đủ bit nhưng HMAC sai (sai khóa hoặc bit bị sửa).
authenticated CAVLC frame version/length is invalid	<code>unpack_frame_with_key()</code> khi khung sai định dạng.
blind schedule capacity is insufficient	Lịch cần nhiều bit hơn số ứng viên.
IDR stream capacity is insufficient for authenticated payload	Embed chế độ file: hết IDR trước khi nhúng đủ khung.
IDR stream ended before authenticated payload capacity was met	Embed live (<code>embed-live-auth-stdin</code>).
IDR stream ended before authenticated payload was complete	Extract chế độ file.

Thông báo	Khi nào / hàm phát sinh
input stream ended before authenticated payload was complete	Extract live (extract-live-auth-stdin).

HMAC hợp lệ chỉ chứng minh payload đến từ người có khóa và không bị sửa; nó **không** thay cho việc kiểm proof Groth16.

8 CLI native và công cụ soi

8.1 zkstego_blind_bits

native/tools/blind_bits.cpp: wmain() (Windows, đối số wide) hoặc main() gọi zkstego::cli::utf8_arguments() rồi run(). Hàm run() dispatch đúng năm lệnh:

Lệnh và đối số	stdin / stdout	Hàm lỗi được gọi
embed-stream-auth-stdin in out max_bits	stdin: dòng khóa hex, dòng payload hex; ghi file mới	embed_authenticated_cavlc_stream_file()
extract-stream-auth in key - max_payload max_bits	khóa từ file quyền hạn chế hoặc stdin (-); stdout: payload hex	extract_authenticated_cavlc_stream_file()
measure-live-capacity- stdin max_bits	stdin: Annex-B; stdout: ZKSTEG_CAPACITY_METRICS JSON	run_live_capacity_measure()
embed-live-auth-stdin max_bits	stdin: khóa, payload, rồi Annex-B; stdout: H.264; stderr: ZKSTEG_STREAM_METRICS	run_live_authenticated_embed()
extract-live-auth-stdin max_payload max_bits	stdin: khóa, rồi Annex-B; stdout: payload hex	run_live_authenticated_extract()

Helper trong blind_bits.cpp	Vai trò
read_key_stdin_line(), read_key_file()	Đúng một dòng 64 ký tự hex; bộ đệm SensitiveKeyText bị xóa khi hủy. Khóa không bao giờ đi qua argv.
read_hex_payload_stdin_line(), decode_hex() parse_bit_count()	Payload hex không rỗng, tối đa 4096 byte; giữ trong SensitiveBytes. Từ chối số âm, tràn, ký tự thừa; các lệnh tự kiểm max_bits > 0.
set_standard_streams_to_binary()	Bắt buộc trên Windows để stdin/stdout không đổi byte H.264.
is_bare_start_code(), parse_single_stream_nal()	Bỏ NAL rỗng ở chế độ live; tách đúng một NAL từ reader.
run_live_authenticated_embed()	Mỗi IDR: assemble_cavlc_stream_segment() → encoder.process_segment() → ghi lại đúng IDR đã patch; SPS/PPS/NAL khác chuyển nguyên; đo p50/p95 thời gian mỗi đoạn.

Helper	Vai trò
<code>run_live_authenticated_extract()</code>	Mỗi IDR: <code>decoder.consume_segment()</code> ; trả ngay khi <code>complete()</code> .

`native/tools/cli_io.hpp` cung cấp `utf8_arguments()`, `os_path()`, `read_binary_file()` (giới hạn `kMaxAnnexBFileBytes = 1 GiB`) và `write_new_binary_file()` (không ghi đè file đã có). `AnnexBNalStreamReader::read_next()` chỉ giữ một NAL cộng một chunk input, nhận start code nằm vắt qua ranh giới chunk, mặc định từ chối NAL lớn hơn 16 MiB.

8.2 zkstego_inspect và lịch tham chiếu Python

`native/tools/inspect.cpp::run()` chọn một trong bốn chế độ: `trace()` (mặc định, JSON schema 1: NAL, slice IDR, ứng viên toàn file), `summary()` (`--summary`, văn bản), `trace_macroblock()` (`--macroblock NAL MB`, schema `macroblock-detail-1`: header mọi MB của slice và từng phần tử CAVLC của một MB) và `segments()` (`--segments MAX_BITS`, schema `segments-1`, dựa trên `analyze_cavlc_stream_file()`).

Python `src.native_blind_contract.segment_schedule()` nhận JSON `segments-1`, khóa, số bit khung và `max_bits_per_IDR`, trả danh sách `SchedulePlacement` (đoạn, ứng viên, chỉ số bit khung) đúng như encoder native. Demo dùng hàm này hai lần: trước khi nhúng (`pack_and_select()`) và sau khi verify (`verify_message()`) để chứng tỏ bên nhận tự dựng lại cùng lịch từ video stego.

Hàm trong <code>src/native_blind_contract.py</code>	Tương ứng native
<code>derive_channel_keys()</code> , <code>hkdf_sha256()</code>	<code>derive_cavlc_subkey()</code> , <code>hkdf_sha256()</code>
<code>NativeCavlcCandidate.serialize()</code> , <code>score_candidate()</code> , <code>select_candidates()</code>	<code>serialize_cavlc_sign_candidate()</code> , <code>score_keyed_cavlc_sign_candidate()</code> , <code>select_keyed_cavlc_sign_candidates()</code>
<code>pack_authenticated_frame()</code> , <code>unpack_authenticated_frame()</code>	<code>pack_authenticated_cavlc_frame()</code> , <code>unpack_authenticated_cavlc_frame()</code>
<code>whitening_keystream_bits()</code> , <code>whiten_bits()</code> (= <code>unwhiten_bits</code>)	<code>cavlc_whitening_keystream()</code> , <code>whiten_cavlc_frame_bits()</code>
<code>embedded_frame_bits()</code> , <code>recover_authenticated_payload()</code>	Bit khung đã làm trắng / chiều ngược
<code>segment_schedule()</code>	Lịch của <code>process_segment()</code> qua mọi đoạn

9 Verifier: từ video stego đến quyết định

9.1 Trích mù native

“Mù” nghĩa là bên nhận không cần video gốc hay file phụ. Với `extract-stream-auth`, native:

1. duyệt NAL, cập nhật ngữ cảnh SPS/PPS, ghép mỗi IDR thành một đoạn;
2. giải mã lại CAVLC, thu ứng viên (giống phía gửi vì lật dấu không đổi cấu trúc);

3. dẫn xuất khóa con, xếp lịch HMAC của đoạn, đọc bit dấu tại vị trí đã chọn;
4. bỏ làm trắng 24 bit đầu, kiểm version `0x02` và độ dài, biết tổng số bit cần đọc;
5. đọc đủ khung qua các đoạn, kiểm tag HMAC; trả payload hoặc thoát mã 2.

9.2 Unpack và verify Groth16

`unpack(blob)` đọc độ dài message big-endian và kiểm blob đủ $4 + |m| + 129$ byte rồi tách (m, proof) . Các caller hiện tại còn đòi độ dài *chính xác* (không dư byte): `verify_message()` trong demo và `make_proof_verify_handler()` trong API. Tiếp theo `bytes_to_proof()` dựng lại điểm Groth16, `_build_public_signals(m, k)` tính hash, commitment và length, rồi `snarkjs verify`. Message chỉ được dùng sau khi proof hợp lệ.

9.3 Verifier phía API

`native_handlers.make_proof_verify_handler()` trả về:

Kết quả	Điều kiện
<code>valid: true, message_b64</code>	Trích được payload, unpack đúng, proof verify. Job → <code>succeeded</code> .
<code>reason: payload_not_authenticated</code>	CLI thoát khác 0 (NativePayloadNotAuthenticated). Job → <code>rejected</code> .
<code>reason: malformed_proof_payload</code>	<code>unpack()</code> / <code>bytes_to_proof()</code> ném <code>ValueError</code> hoặc độ dài lệch.
<code>reason: proof_invalid</code>	Groth16 trả <code>False</code> .
Ngoại lệ hạ tầng	Timeout, thiếu Node.js hoặc verification key → job <code>failed</code> , không phải <code>rejected</code> .

10 API: hàm nào nhận request, hàm nào chạy job

10.1 Lỗi job chung `src/api/app.py`

`create_app(settings, embed_handler=, verify_handler=, extract_handler=)` đăng ký endpoint; endpoint có handler `None` trả 501. Ứng dụng `native inject` đủ cả ba.

Hàm/lớp	Input → output	Ảnh hưởng
<code>ApiSettings.from_environment()</code>	<code>env</code> → <code>settings</code>	<code>ZK_STEGO_API_TOKEN</code> bắt buộc, tối thiểu 32 ký tự (<code>MIN_API_TOKEN_LENGTH</code>); <code>work dir</code> , <code>circuits dir</code> , <code>timeout upload</code> , thời gian giữ job, <code>ZK_STEGO_API_DOCS</code> .
<code>RequestBodyGuard.__call__()</code>	ASGI scope → request được bảo vệ	Kiểm Bearer token trước khi đọc body , giới hạn kích thước/thời gian upload, tối đa 4 upload đồng thời.

Hàm/lớp	Input → output	Ảnh hưởng
AuthFailureLimiter	IP → retry_after()	10 lần sai trong 60 s → 429 kèm Retry-After; dùng chung cho HTTP và WebSocket.
bearer_token_matches(), require_token()	header → pass/401	So token; require_token() là lớp phòng thủ thứ hai.
WorkDirLease.acquire()/release()	work dir → khóa	Ngăn hai instance dùng chung thư mục.
JobStore.create/get/update/discard()	record ↔ JSON	Không lưu secret hay nội dung message.
_decode_key(), _decode_message()	base64 → bytes	Kiểm định dạng trước khi xếp job.
_save_upload()	UploadFile → file	Ghi có giới hạn; lỗi thì dọn.
reserve_capacity()	semaphore → slot/503	Mặc định 1 worker + 2 job chờ.
submit()	job id, operation, callable	Chạy nền; job verify có valid khác True → rejected; ngoại lệ → failed.
start_embed()	video, message_b64, secret_key_b64	Gọi embed_handler.
start_extract()	stego_video, khóa, maximum_payload_bytes (1.4096)	Gọi extract_handler; artifact payload.bin.
start_verify()	stego_video, khóa, maximum_payload_bytes	Gọi verify_handler; không cần video gốc.
get_job(), download_artifact()	job id → trạng thái/file	Artifact tải một lần , hết hạn sau 10 phút.
startup(), shutdown(), recover_interrupted_jobs(), purge_expired_jobs()	vòng đời	Khôi phục job đang dở, dọn job quá hạn (mặc định 24 h).

10.2 Ứng dụng native src/api/native_handlers.py

create_native_app() đọc ZK_STEGO_NATIVE_CLI (đường dẫn zkstego_blind_bits) và ZK_STEGO_MAX_BITS_PER_IDR (mặc định 64), rồi định nghĩa:

- `invoke()`: chạy CLI không qua shell, có timeout; stdin = khóa hex + phần đuôi; mã thoát khác 0 → `NativePayloadNotAuthenticated`.
- `embed_handler()`: `embed-stream-auth-stdin`; `bits_embedded` được tính là $(|m| + 19) \times 8$ (`bits_embedded_source = computed_frame_size`), không do CLI báo.
- `native_extract_payload()` và `extract_handler()`: `extract-stream-auth ...` -, kiểm hex và giới hạn payload, ghi file mới.
- `groth16_verify()`: `ZKSnarkBridge(settings.circuits_dir)`; thiếu verification key là lỗi triển khai (failed).
- `native_stream()`: `WebSocket /api/v1/stream`.

Giao thức WebSocket: xác thực *trước* `accept()` (thiếu token thì handshake bị từ chối). Trình tự thông điệp:

```

client -> {"operation":"embed","secret_key_b64":...,"message_b64":...}
        or {"operation":"extract","secret_key_b64":...,"maximum_payload_bytes":N}
server -> {"type":"ready","operation":...}
client -> binary Annex-B chunks ..., then {"type":"end"}
server -> embed: binary H.264 chunks, then {"type":"complete",...}
        extract: {"type":"payload","payload_b64":...}
        error: {"type":"error","code":"payload_not_authenticated" | "
stream_idle_timeout" | ...}

```

Server giới hạn byte/chunk, đặt high/low water mark stdin của tiến trình con (64/16 KiB), đọc stderr liên tục và đóng luồng sau ZK_STEGO_STREAM_IDLE_TIMEOUT_SECONDS (15 s) không có dữ liệu. Đây là dịch vụ trên elementary stream H.264; module này không tự mở thiết bị camera.

11 Các module hỗ trợ ngoài đường embed/extract

11.1 Bộ lập lịch realtime

src/realtime_cavlc.py: CAVLCRealtimeBudget.__post_init__() kiểm FPS, giới hạn hàng đợi và bộ nhớ; RealtimeCAVLCScheduler.submit() nhận một đoạn với epoch tăng đơn điệu, bỏ đoạn cũ nhất khi đầy; process_next() lấy proof qua _proof_for() (cache tối đa 4 epoch), gọi callback patch, đo độ trễ; report() tính số đoạn bị bỏ, failed_segments, p95 và accepted. Đây là abstraction được test riêng; WebSocket native tự quản lý tiến trình con và backpressure, không gọi scheduler này. Đo trên camera thật hiện **chưa** đạt cổng 30 FPS.

11.2 Chứng chỉ khóa và manifest

src/key_policy.py::issue_key_certificate() ký chứng chỉ Ed25519 (khóa công khai chủ thể, key id, issuer, thời điểm bắt đầu/hết hạn); KeyCertificate.verify() kiểm chữ ký, định dạng và cửa sổ thời gian; verify_manifest_with_certificate() kiểm chứng chỉ rồi verify manifest. src/manifest.py::StegoManifest có sign(), verify_signature(), compute_content_hash(), to_json()/from_json(). Các helper này chỉ có hiệu lực nếu được gọi tại điểm chấp nhận khóa; đường native hiện nhận khóa thô 32 byte và không gọi KeyCertificate.verify().

11.3 Giao diện trust thử nghiệm

src/trust/ gồm provenance.py, fingerprint.py, attestation.py, watermark_receipt.py, canonical.py và c2pa_bridge.py (build_c2pa_anchor(), verify_c2pa_anchor()). Các mạch circuits/fingerprint_verify.circom và circuits/detector_receipt.circom thuộc cùng hướng thử nghiệm. Chúng được kiểm bởi test_trust_interfaces.py nhưng không nằm trên đường runtime của demo hay API.

12 Kiểm thử và tái lập

```

py -3.12 -m pip install -r requirements.txt
cd circuits; npm ci; cd ..
cmake --build native/build --config Release
py -3.12 src/runtest/prepare_fixtures.py      # regenerate data/encoded/*.h264 from
data/raw
py -3.12 src/runtest/run_all.py              # all 12 phases
py -3.12 src/runtest/run_all.py --quick     # fast phases only
ctest --test-dir native/build -C Release --output-on-failure

```

Phase	File trong src/runtest/	Nội dung
1	test_zk_proof.py	Proof Groth16: định dạng, nén, verify, fail-closed
2	test_native_blind_contract.py	C++ và Python khớp HKDF, lịch, khung, làm trắng, lịch theo đoạn
3	test_native_cli_fixture.py	CLI nhúng/trích trên fixture, decode nghiêm ngặt, sai khóa
4	test_h264_tables.py	Bảng VLC (prefix-free, Kraft, giá trị chuẩn)
5	test_service_api.py	Lỗi job, xác thực trước body, giới hạn thử sai, job verify
6	test_native_http_channel.py	Job HTTP và WebSocket qua Uvicorn thật
7	test_manifest_security.py	Ký manifest và ràng buộc liên quan
8	test_realtime_scheduler.py	Bộ lập lịch realtime
9	test_benchmark_recorder.py	Fail-closed của bộ ghi benchmark camera
10	test_trust_interfaces.py	Provenance, C2PA, attestation thử nghiệm
11	test_demo_h264_explain.py	Toán giải thích của demo khớp native và pixel Ffmpeg
12	test_demo_smoke.py	Toàn bộ demo chạy không có [FAIL]
H1	test_native_camera_http.py	Camera thật, chỉ khi --hardware

Mã thoát của run_all.py: 0 = tất cả qua, 1 = có lỗi, 2 = có phase thiếu bằng chứng (bị bỏ qua). Theo README.md, lần chạy ngày 03/10/2026 đạt 126/126 qua 12 phase và CTest native 1/1; phase camera H1 không nằm trong con số này.

13 Demo đã chạy: artifact và cách suy luận

Artifact	Nội dung	Dùng để kết luận gì
transcript.txt, report.json	Thuyết minh (secret bị che), kết quả kiểm tra, thời gian	Trạng thái tổng; luôn mở log tiến trình gốc khi có lỗi.
encode.stderr, encoder_input.yuv, source_decoded.yuv	Log Ffmpeg/libx264, frame trước/sau nén	Mất mát do nén, tách biệt với nhúng.
trace_before.json, trace_after.json	NAL, slice, ứng viên, bit dấu, hệ số	Chỉ đúng các bit đã lập lịch đổi; cấu trúc giữ nguyên.
segments_before.json, segments_after.json	Ứng viên từng đoạn (segments-1)	Bên nhận dựng lại cùng lịch từ stego.
schedule.json	Bit khung, keystore, bit nhúng, cũ/mới từng vị trí	Kiểm lịch HMAC và làm trắng.
proof.json, proof.bin, public_signals.json	Proof và 513 public signal	Kiểm nén/giải nén proof.
mb_detail_before.json, mb_detail_after.json	Output --macroblock	Giải phẫu khối mẫu.
native_embed.*, native_extract.*, reject_*.*	stdout/stderr từng tiến trình	Mã thoát 2 và thông báo lỗi đúng lý do.

PASS của demo là bằng chứng chức năng trên một clip/một lần chạy, không phải benchmark: nó không đo chất lượng cảm nhận trên tập lớn và không chứng minh deadline realtime.

14 Các giới hạn cần nhớ khi đọc code

1. **Không phải mọi H.264 đều được hỗ trợ.** Parser native chỉ nhận Baseline, CAVLC, progressive, 4:2:0, một slice mỗi frame bắt đầu ở MB 0, không I_PCM, không nhiều SPS/PPS khác id.
2. **Chỉ IDR mang bit.** Với GOP > 1, P-frame được sao chép nguyên.
3. **Mù không có nghĩa không cần khóa.** Extractor không cần video gốc nhưng cần khóa và cùng max_bits_per_IDR.
4. **Groth16, HMAC và bearer token khác nhiệm vụ.** Proof kiểm quan hệ circuit; HMAC kiểm khung theo khóa chung; token xác thực caller của API.
5. **Không chống phát lại.** Khung và proof không chứa định danh video hay bộ đếm; payload có thể bị chép sang video khác dùng cùng khóa (xem future_plan.md).
6. **Ai có khóa cũng tạo được proof,** vì verifier phải biết secret để tính lại commitment.

15 Lộ trình đọc source cho người mới tiếp nhận

1. Đọc doc/he_thong_hoat_dong.md và demo/README.md, chạy `py -3.12 demo/terminal_demo.py --auto`, mở thư mục `demo/runs/terminal_*`.
2. Đọc `terminal_demo.py::walkthrough()` theo bảng ở Mục 2.2, rồi `demo/native_io.py` để thấy Python chỉ gọi tiến trình native.
3. Đọc `native/tools/blind_bits.cpp::run()`, sau đó `embed_authenticated_cavlc_stream_file()` và `extract_authenticated_cavlc_stream_file()`.
4. Đọc thư viện theo chuỗi `split_annex_b() → decode_baseline_i_idr_slices() → collect_cavlc_trailing_one_sign_candidates() → prepare_cavlc_stream_segment() → select_with_schedule_key() → AuthenticatedCavlcStreamEncoder/Decoder`.
5. Đối chiếu với `src/native_blind_contract.py` và `test_native_blind_contract.py`.
6. Đọc proof path: `src/zk_proof.py` và `circuits/payload_verify.circom`.
7. Đọc boundary dịch vụ: `native_handlers.create_native_app()`, `make_proof_verify_handler()`, `native_stream()` và `app.create_app()`.

16 Tra cứu nhanh theo câu hỏi debug

Triệu chứng	Nơi bắt đầu lần theo
Không thấy IDR / parser từ chối	<code>zkstego_inspect --summary; split_annex_b(), parse_baseline_sps()/pps(), decode_baseline_i_idr_slices();</code> kiểm profile và <code>slices=1</code> .
Dung lượng thấp, payload không vừa	<code>zkstego_inspect --segments N; segment_plan.resolve_max_bits();</code> tăng <code>max_bits_per_IDR</code> , số frame hoặc giảm message.
Embed thoát mã 2	<code>stderr</code> của <code>native_embed;</code> <code>embed_authenticated_cavlc_stream_file(), process_segment().</code>
FFmpeg từ chối video	Kiểm ranh giới patch/EBSP: <code>patch_annex_b_rbsp_plan(), rbsp_to_esp().</code>

Triệu chứng	Nơi bắt đầu lần theo
Extract báo sai version/tag	Cùng khóa hai phía? Cùng max_bits_per_IDR? Rồi consume_segment(), unpack_frame_with_key().
Lịch Python khác native	segment_schedule() so với analyze_cavlc_stream_file(); nhớ nal_index tính trong input phân tích của đoạn.
Payload trích được nhưng ZK false	Kiểm 4 byte độ dài, proof 129 byte, public signals và verification_key.json; xem _snarkjs_verify().
Job verify rejected	Đọc reason: payload_not_authenticated, malformed_proof_payload, proof_invalid.
Job verify failed	Lỗi hạ tầng: Node.js, ZK_STEGO_CIRCUITS_DIR, verification key, timeout.
HTTP 401/429	RequestBodyGuard, AuthFailureLimiter; token phải ≥ 32 ký tự.
Khóa hết hạn vẫn dùng được	Đường native nhận khóa thô; KeyCertificate.verify() chưa được gọi ở đó.

17 Kết luận vận hành

Khi đọc một đường lỗi, lần theo artifact tuần tự: message, proof 129 byte, payload, khung v2 (đã làm trắng), lịch theo đoạn, video stego, payload trích được, message và proof sau unpack, rồi kết quả Groth16. Điểm nổi quan trọng nhất là CLI native nhận payload là byte opaque: Python tạo proof và payload; phân tích cú pháp H.264, khóa con, lịch, khung, làm trắng, patch và trích mù đều nằm trong C++. Khi sửa một hàm, kiểm đúng consumer kế tiếp của nó và chạy test ở boundary tương ứng (CTest cho lỗi, phase 2 cho hợp đồng chéo, phase 3/5/6 cho CLI và dịch vụ).

Ghi chú lịch sử. Trước ngày 03/10/2026, repository còn một nhánh Python full-video (src/embedder.py, src/verifier*.py, src/core/, src/bitstream/, chaos map, sidecar positions.json, cache pickle) và bộ demo nhiều script đánh số; nhánh này và các benchmark SEC1–SEC10 đã bị gỡ (vẫn xem được trong lịch sử git), còn c2pa_bridge.py được chuyển từ src/provenance/ sang src/trust/.